

BABU BANARASI DAS UNIVERSITY

LUCKNOW, U.P



Lab Manual for – Compiler Design Lab

NCS4654

Submitted by:

Shivam Singh (1230439285)

Submitted to:

Ms. Mala Chaturvedi

INDEX

[illegible]

Program – 1

Implementation of Lexical Analyzer for IF Statement

THEORY

Aim

To design and implement a **Lexical Analyzer** that identifies tokens in an if statement.

Introduction

A **Lexical Analyzer (Scanner)** is the first phase of a compiler.

It reads the source program character by character and converts it into a stream of tokens.

It removes:

- White spaces
- New lines
- Comments

And generates:

- Keywords
- Identifiers
- Numbers
- Operators
- Delimiters

Role in Compiler Structure

Source Program → Lexical Analyzer → Tokens → Syntax Analyzer → Parse Tree

Algorithm

Step 1: Start

Begin lexical analysis process.

Step 2: Initialization

- Load source program into input buffer
- Initialize pointers (begin, forward)
- Initialize token table and symbol table

Step 3: Read Character

Read next character using forward pointer.

Step 4: Skip White Spaces & Comments

Ignore blanks, tabs, new lines, comments.

Step 5: Pattern Matching

Match input characters with token patterns.

Step 6: Token Recognition

If matched:

- Keyword → Generate keyword token
- Identifier → Generate identifier token
- Number → Generate number token
- Operator/Delimiter → Generate respective token

Step 7: Symbol Table Handling

- If identifier is new → Add to symbol table
- Store attributes if needed

Step 8: Error Handling

If no pattern matches → Generate error token.

Step 9: Return Token

Return token to syntax analyzer.

Step 10: Repeat

Repeat until end of file.

Step 11: Stop

Terminate lexical analyzer.

Tokens Recognized

Token	Example
IF	if
IDENTIFIER	x, y
NUMBER	10, 5
OPERATOR	+, -, >
ASSIGN	=
LPAREN	(
RPAREN	)
LBRACE	{
RBRACE	}
SEMICOLON	;

LEX TOOL CODE (Using Flex)

Below is the equivalent Lex program for the same lexical analyzer:

```
%{
#include <stdio.h>
%}

%%

"if"      { printf("IF\t\t%s\n", yytext); }
[0-9]+    { printf("NUMBER\t\t%s\n", yytext); }
[a-zA-Z_][a-zA-Z0-9_]* { printf("IDENT\t\t%s\n", yytext); }
"("       { printf("LPAREN\t\t%s\n", yytext); }
")"       { printf("RPAREN\t\t%s\n", yytext); }
"{"       { printf("LBRACE\t\t%s\n", yytext); }
"}"       { printf("RBRACE\t\t%s\n", yytext); }
"="       { printf("ASSIGN\t\t%s\n", yytext); }
";"       { printf("SEMICOLON\t\t%s\n", yytext); }
[+ \- * / > <] { printf("OPERATOR\t\t%s\n", yytext); }
[ \t\n]+  ; /* Ignore whitespace */
.         { printf("UNKNOWN\t\t%s\n", yytext); }
%%

int main() {
    printf("Token Type\tValue\n");
    printf("-----\n");
```

C IMPLEMENTATION

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>


/* Token types */
typedef enum {
    TOKEN_IF,
    TOKEN_IDENTIFIER,
    TOKEN_NUMBER,
    TOKEN_OPERATOR,
    TOKEN_LPAREN,
    TOKEN_RPAREN,
    TOKEN_LBRACE,
    TOKEN_RBRACE,
    TOKEN_ASSIGN,
    TOKEN_SEMICOLON,
    TOKEN_END,
    TOKEN_UNKNOWN
} TokenType;


/* Token structure */
typedef struct {
    TokenType type;
```

```
char value[64];

} Token;

/* Convert token type to string */
const char* tokenTypeToString(TokenType type) {
    switch (type) {
        case TOKEN_IF: return "IF";
        case TOKEN_IDENTIFIER: return "IDENT";
        case TOKEN_NUMBER: return "NUMBER";
        case TOKEN_OPERATOR: return "OPERATOR";
        case TOKEN_LPAREN: return "LPAREN";
        case TOKEN_RPAREN: return "RPAREN";
        case TOKEN_LBRACE: return "LBRACE";
        case TOKEN_RBRACE: return "RBRACE";
        case TOKEN_ASSIGN: return "ASSIGN";
        case TOKEN_SEMICOLON: return "SEMICOLON";
        case TOKEN_END: return "END";
        default: return "UNKNOWN";
    }
}

/* Get next token */
Token getNextToken(const char **input) {
    Token token;
    token.value[0] = '\0';
```



```
/* Skip whitespace */
while (isspace(**input))
    (*input)++;

/* End of input */
if (**input == '\0') {
    token.type = TOKEN_END;
    return token;
}

/* IF keyword */
if (strncmp(*input, "if", 2) == 0 && !isalnum((*input)[2])) {
    token.type = TOKEN_IF;
    strcpy(token.value, "if");
    *input += 2;
}

/* Parentheses */
else if (**input == '(') {
    token.type = TOKEN_LPAREN;
    strcpy(token.value, "(");
    (*input)++;
}

else if (**input == ')') {
    token.type = TOKEN_RPAREN;
    strcpy(token.value, ")");
    (*input)++;
}
```

```
}

/* Braces */
else if (**input == '{') {
    token.type = TOKEN_LBRACE;
    strcpy(token.value, "{");
    (*input)++;
}

else if (**input == '}') {
    token.type = TOKEN_RBRACE;
    strcpy(token.value, "}");
    (*input)++;
}

/* Assignment */
else if (**input == '=') {
    token.type = TOKEN_ASSIGN;
    strcpy(token.value, "=");
    (*input)++;
}

/* Semicolon */
else if (**input == ';') {
    token.type = TOKEN_SEMICOLON;
    strcpy(token.value, ";");
    (*input)++;
}

/* Identifier */
else if (isalpha(**input) || **input == '_') {
```

```
const char *start = *input;

while (isalnum(**input) || **input == '_')
    (*input)++;

strncpy(token.value, start, *input - start);
token.value[*input - start] = '\0';
token.type = TOKEN_IDENTIFIER;
}

/* Number */
else if (isdigit(**input)) {
    const char *start = *input;
    while (isdigit(**input))
        (*input)++;

    strncpy(token.value, start, *input - start);
    token.value[*input - start] = '\0';
    token.type = TOKEN_NUMBER;
}

/* Operators */
else if (strchr("+-* /><", **input)) {
    token.type = TOKEN_OPERATOR;
    token.value[0] = **input;
    token.value[1] = '\0';
    (*input)++;
}

/* Unknown */
```

```
else {  
    token.type = TOKEN_UNKNOWN;  
    token.value[0] = **input;  
    token.value[1] = '\\0';  
    (*input)++;  
}  
  
return token;  
}  
  
int main() {  
    const char *code = "if (x > 10) { y = x + 5; }";  
    const char *input = code;  
    Token token;  
  
    printf("Token Type  Value\\n");  
    printf("----- ----\\n");  
  
    while ((token = getNextToken(&input)).type != TOKEN_END) {  
        printf("%-12s %s\\n", tokenTypeToString(token.type), token.value);  
    }  
  
    return 0;  
}
```

Output - 1

```
shiva@Asus MINGW64 /c/Compiiler design Lab
$ gcc Program1.c -o Program1.exe
./Program1.exe
Token Type    Value
-----
IF            if
LPAREN        (
IDENT         x
OPERATOR      >
NUMBER        10
RPAREN        )
LBRACE        {
IDENT         y
ASSIGN        =
IDENT         x
OPERATOR      +
NUMBER        5
SEMICOLON     ;
RBRACE        }
```

Program - 2

Implementation of Lexical Analyzer for Arithmetic Expression

THEORY

Aim

To design and implement a Lexical Analyzer that identifies tokens in an arithmetic expression.

Introduction

A Lexical Analyzer is the first phase of a compiler. It scans the input source code character by character and converts it into meaningful tokens.

For arithmetic expressions, the lexical analyzer identifies:

- Numbers
- Operators (+, -, *, /)
- Parentheses
- Invalid symbols

It removes unnecessary white spaces and returns tokens to the syntax analyzer.

Role in Compilation Process

Arithmetic Expression → Lexical Analyzer → Token Stream → Parser

The lexical analyzer:

- Groups characters into lexemes
- Classifies lexemes into token types
- Detects invalid characters
- Prepares input for syntax analysis

Algorithm

Step 1: Start

Begin the lexical analysis process.

Step 2: Initialization

- Load the source program into input buffer
- Initialize begin and forward pointers
- Initialize token and symbol tables

Step 3: Read Character

Read next character using forward pointer.

Step 4: Skip White Spaces & Comments

Ignore blanks, tabs, and new lines.

Step 5: Pattern Matching

Match input characters with predefined patterns.

Step 6: Token Recognition

If pattern matches:

- Number → Generate NUMBER token
- Operator → Generate OPERATOR token
- Left parenthesis → Generate LPAREN token
- Right parenthesis → Generate RPAREN token

Step 7: Symbol Table Handling

(Not mandatory here since arithmetic expression contains only numbers and operators.)

Step 8: Error Handling

If character does not match any pattern → Generate UNKNOWN token.

Step 9: Return Token

Return token to syntax analyzer.

Step 10: Repeat

Repeat until end of input.

Step 11: Stop

Terminate lexical analyzer.

Tokens Recognized

Token Type Example

NUMBER 10, 25

OPERATOR +, -, *, /

LPAREN (

RPAREN)

UNKNOWN @, #

LEX (FLEX) TOOL CODE

Below is the equivalent Lex implementation:

```
%{  
#include <stdio.h>  
%}  
  
%%  
[0-9]+      { printf("NUMBER\t\t%s\n", yytext); }  
[+ \- * /]  { printf("OPERATOR\t\t%s\n", yytext); }  
"("         { printf("LPAREN\t\t%s\n", yytext); }  
")"         { printf("RPAREN\t\t%s\n", yytext); }  
[ \t\n]+    ; /* Ignore whitespace */  
.  
            { printf("UNKNOWN\t\t%s\n", yytext); }  
%%  
  
int main() {  
    printf("Token Type\tValue\n");  
    printf("-----\n");  
    yylex();  
    return 0;  
}
```

C IMPLEMENTATION

```
#include <stdio.h>

#include <ctype.h>

#include <string.h>


/* Token types */
typedef enum {
    TOKEN_NUMBER,
    TOKEN_OPERATOR,
    TOKEN_LPAREN,
    TOKEN_RPAREN,
    TOKEN_UNKNOWN,
    TOKEN_END
} TokenType;


/* Token structure */
typedef struct {
    TokenType type;
    char value[64];
} Token;


/* Convert token type to string */
const char* tokenTypeToString(TokenType type) {
    switch (type) {
```

```
case TOKEN_NUMBER: return "NUMBER";
case TOKEN_OPERATOR: return "OPERATOR";
    case TOKEN_LPAREN: return "LPAREN";
    case TOKEN_RPAREN: return "RPAREN";
    case TOKEN_UNKNOWN: return "UNKNOWN";
    case TOKEN_END: return "END";
    default: return "UNKNOWN";
}
}
```

```
/* Get next token */
```

```
Token getNextToken(const char **input) {
```

```
    Token token;
```

```
    token.value[0] = '\0';
```

```
    while (isspace(**input))
```

```
        (*input)++;
```

```
    if (**input == '\0') {
```

```
        token.type = TOKEN_END;
```

```
        return token;
```

```
    }
```

```
    if (isdigit(**input)) {
```

```
        const char *start = *input;
```

```
        while (isdigit(**input))
```

```
    (*input)++;

    strncpy(token.value, start, *input - start);
    token.value[*input - start] = '\0';
    token.type = TOKEN_NUMBER;
}
else if (strchr("+-*/", **input)) {
    token.type = TOKEN_OPERATOR;
    token.value[0] = **input;
    token.value[1] = '\0';
    (*input)++;
}
else if (**input == '(') {
    token.type = TOKEN_LPAREN;
    strcpy(token.value, "(");
    (*input)++;
}
else if (**input == ')') {
    token.type = TOKEN_RPAREN;
    strcpy(token.value, ")");
    (*input)++;
}
else {
    token.type = TOKEN_UNKNOWN;
    token.value[0] = **input;
    token.value[1] = '\0';
```

```

(*input)++;

}

    return token;
}

int main() {
    char code[200];
    const char *input;
    Token token;

    printf("Enter an arithmetic expression: ");
    fgets(code, sizeof(code), stdin);

    input = code;

    printf("\nToken Type    Value\n");
    printf("-----\n");

    while ((token = getNextToken(&input)).type != TOKEN_END) {
        printf("%-15s %s\n", tokenTypeToString(token.type), token.value);
    }

    return 0;
}

```

Output - 2

```
shiva@Asus MINGW64 /c/Compiiler design Lab
● $ gcc Program2.c -o Program2.exe
  ./Program2.exe
  Enter an arithmetic expression: 4+6*(7/7)-11
```

Token Type	Value
-----	-----
NUMBER	4
OPERATOR	+
NUMBER	6
OPERATOR	*
LPAREN	(
NUMBER	7
OPERATOR	/
NUMBER	7
RPAREN	)
OPERATOR	-
NUMBER	11

Program – 3

Implementation of Shift Reduce Parsing Algorithm

THEORY

Aim

To implement a Shift Reduce Parser for validating arithmetic expressions using a stack-based bottom-up parsing technique.

Introduction

A Shift Reduce Parser is a type of Bottom-Up Parser used in compiler design. It constructs the parse tree from leaves (input symbols) to the root (start symbol).

It uses:

- A stack
- An input buffer
- A set of grammar productions

It performs two main operations:

1. Shift – Push next input symbol onto stack
2. Reduce – Replace RHS of a production with its LHS

Grammar Used

The parser follows the standard expression grammar:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$

$F \rightarrow id$

Where:

- $E \rightarrow \text{Expression}$
- $T \rightarrow \text{Term}$
- $F \rightarrow \text{Factor}$
- $id \rightarrow \text{Identifier}$

Shift Reduce Parsing Algorithm

1. Push \$ onto stack
2. Append \$ to input string
3. While (true):
4. If stack top matches RHS of production $\rightarrow$ Reduce
5. Else if input not empty $\rightarrow$ Shift
6. Else if stack = \$S and input = \$ $\rightarrow$ Accept
7. Else $\rightarrow$ Reject
8. End

Working Principle

Operation Description

Shift	Moves input symbol to stack
Reduce	Replaces RHS with LHS
Accept	Input is valid
Reject	Input is invalid

YACC TOOL CODE (Shift Reduce Parser)

Below is equivalent YACC implementation:

```
%{  
#include <stdio.h>  
#include <stdlib.h>  
%}  
  
%token ID  
  
%%  
  
E : E '+' T  
  | T  
  ;  
  
T : T '*' F  
  | F  
  ;  
  
F : ID  
  ;  
  
%%
```

```
int main() {  
    printf("Enter expression:\n");  
    yyparse();  
    return 0;  
}
```

```
int yyerror() {  
    printf("Parsing failed\n");  
    return 0;  
}
```

Lex File (For ID token)

```
%{  
#include "y.tab.h"  
%}  
  
%%  
[a-zA-Z][a-zA-Z0-9]* { return ID; }  
[ \t\n]+ ;  
. { return yytext[0]; }  
%%
```

C IMPLEMENTATION

```
#include <stdio.h>

#include <stdlib.h>

#include <string.h>

#include <ctype.h>


#define MAX_STACK_SIZE 100

#define MAX_INPUT_SIZE 100


typedef struct {

    char stack[MAX_STACK_SIZE];

    int top;

} Stack;


void initStack(Stack *s) {

    s->top = -1;

}


void push(Stack *s, char c) {

    if (s->top < MAX_STACK_SIZE - 1) {

        s->stack[++(s->top)] = c;

    } else {

        printf("Stack overflow\n");

        exit(1);

    }

}
```

```
}
```

```
char pop(Stack *s) {
```

```
    if (s->top >= 0) {
```

```
        return s->stack[(s->top)--];
```

```
    } else {
```

```
        printf("Stack underflow\n");
```

```
        exit(1);
```

```
    }
```

```
}
```

```
char peek(Stack *s) {
```

```
    if (s->top >= 0)
```

```
        return s->stack[s->top];
```

```
    else
```

```
        return '\0';
```

```
}
```

```
void printStack(Stack *s) {
```

```
    for (int i = 0; i <= s->top; i++)
```

```
        printf("%c", s->stack[i]);
```

```
    printf("\n");
```

```
}
```

```
int matchReduceRule(Stack *s, const char *rule) {
```

```
    int ruleLength = strlen(rule);
```

```
if (ruleLength > s->top + 1)
    return 0;

for (int i = 0; i < ruleLength; i++) {
    if (s->stack[s->top - ruleLength + 1 + i] != rule[i])
        return 0;
}
return 1;
}
```

```
void shiftReduceParse(const char *input) {
```

```
    Stack s;
```

```
    initStack(&s);
```

```
    const char *current = input;
```

```
    printf("Initial input: %s\n", input);
```

```
    while (*current != '\0') {
```

```
        printf("Shift: %c\n", *current);
```

```
        push(&s, *current);
```

```
        current++;
```

```
        printStack(&s);
```

```
        while (matchReduceRule(&s, "id")) {
```

```
            printf("Reduce by F -> id\n");
```

```
    pop(&s); pop(&s);

    push(&s, 'F');

    printStack(&s);
}

while (matchReduceRule(&s, "F")) {

    printf("Reduce by T -> F\n");

    pop(&s);

    push(&s, 'T');

    printStack(&s);
}

while (matchReduceRule(&s, "T*F")) {

    printf("Reduce by T -> T * F\n");

    pop(&s); pop(&s); pop(&s);

    push(&s, 'T');

    printStack(&s);
}

while (matchReduceRule(&s, "E+T")) {

    printf("Reduce by E -> E + T\n");

    pop(&s); pop(&s); pop(&s);

    push(&s, 'E');

    printStack(&s);
}
```

```
while (matchReduceRule(&s, "T")) {  
    printf("Reduce by E -> T\n");  
    pop(&s);  
    push(&s, 'E');  
    printStack(&s);  
}  
  
}  
  
if (peek(&s) == 'E' && s.top == 0)  
    printf("Parsing successful\n");  
else  
    printf("Parsing failed\n");  
}  
  
int main() {  
    const char *input = "id+id*id";  
    shiftReduceParse(input);  
    return 0;  
}
```

Output - 3

```
shiva@Asus MINGW64 /c/Compiiler design Lab
$ gcc Program3.c -o Program3.exe
./Program3.exe
Initial input: id+id*id
Shift: i
i
Shift: d
id
Reduce by E -> id
E
Shift: +
E+
Shift: i
E+i
Shift: d
E+id
Reduce by E -> id
E+E
Reduce by E -> E+E
E
Shift: *
E*
Shift: i
E*i
Shift: d
E*id
Reduce by E -> id
E*E
Reduce by E -> E*E
E
Parsing successful
```


Program – 4

Implementation of LALR Parser

THEORY

Aim

To implement an **LALR (Look Ahead LR) Parser** for arithmetic expressions using parsing tables.

Introduction

An **LALR Parser** is a bottom-up parsing technique used in compiler design.

It is a simplified version of **CLR (Canonical LR)** parser.

LALR combines:

- LR(0) item sets
- Lookahead symbols

It uses:

- Stack
- ACTION table
- GOTO table
- Input buffer

LALR parsers are widely used because:

- Smaller parsing tables than CLR
- Same power as CLR in most practical grammars
- Used by tools like **YACC/Bison**

Grammar Used (Arithmetic Expression)

$E \rightarrow E + T$

$E \rightarrow T$

$T \rightarrow T * F$

$T \rightarrow F$

$F \rightarrow \text{id}$

LALR Parsing Algorithm

1. Initialize stack with state 0
2. Append \$ to input string
3. Let a be first input symbol

Repeat:

- Let s = top of stack
- If ACTION[s,a] = shift t
→ Push a, push t, move input pointer
- If ACTION[s,a] = reduce $A \rightarrow \beta$
→ Pop $2 \times |\beta|$ symbols
→ Let t = new top
→ Push A
→ Push GOTO[t,A]
- If ACTION[s,a] = accept
→ Print "Accepted"
- Else
→ Print "Syntax Error"

Components of LALR Parser

Component Function

Stack	Stores states and symbols
ACTION Table	Determines shift/reduce/accept
GOTO Table	Determines next state
Input Buffer	Stores input string

Advantages

- Efficient
- Detects syntax errors
- Table-driven parsing
- Industry standard (used in YACC)

YACC IMPLEMENTATION (Real LALR Parser)

YACC automatically generates an **LALR parser**.

parser.y (YACC File)

```
%{  
  
#include <stdio.h>  
  
#include <stdlib.h>  
  
%}  
  
%token NUMBER
```

%%

E : E '+' T

| T

;

T : T '*' F

| F

;

F : NUMBER

;

%%

```
int main() {
```

```
    printf("Enter arithmetic expression:\n");
```

```
    yyparse();
```

```
    return 0;
```

```
}
```

```
int yyerror() {
```

```
    printf("Syntax Error\n");
```

```
    return 0;
```

```
Error\n");
```

```
    return 0;
```

lexer.l (Lex File)

```
%{
```

```
#include "y.tab.h"
```

```
%}
```

```
%%
```

```
[0-9]+    { return NUMBER; }
```

```
[ \t\n]+  ;
```

```
.        { return yytext[0]; }
```

```
%%
```

C IMPLEMENTATION

```
#include <stdio.h>

#include <stdlib.h>

#include <ctype.h>


#define STACK_SIZE 100

#define INPUT_SIZE 100


typedef struct {
    int stack[STACK_SIZE];
    int top;
} Stack;


void initStack(Stack *s) {
    s->top = -1;
}


void push(Stack *s, int value) {
    if (s->top < STACK_SIZE - 1)
        s->stack[++(s->top)] = value;
    else {
        printf("Stack overflow\n");
        exit(1);
    }
}
```

```
int pop(Stack *s) {  
    if (s->top >= 0)  
        return s->stack[(s->top)--];  
    else {  
        printf("Stack underflow\n");  
        exit(1);  
    }  
}
```

```
int peek(Stack *s) {  
    if (s->top >= 0)  
        return s->stack[s->top];  
    return -1;  
}
```

```
#define SHIFT 1  
#define REDUCE 2  
#define ACCEPT 3  
#define ERROR 4
```

```
int parseTable[5][3] = {  
    {SHIFT, ERROR, SHIFT},  
    {ERROR, ERROR, ERROR},  
    {REDUCE, REDUCE, REDUCE},  
    {SHIFT, ERROR, ERROR},  
    {REDUCE, REDUCE, REDUCE}
```

```
};
```

```
void parseError(const char *message) {  
    printf("Error: %s\n", message);  
    exit(1);  
}
```

```
void parse(const char *input) {  
    Stack stack;  
    initStack(&stack);  
  
    int state = 0;  
    int action;  
    int index = 0;  
    char currentChar;  
  
    while ((currentChar = input[index++]) != '\0') {  
  
        if (currentChar == '\n') break;  
  
        int charIndex;  
  
        if (currentChar == '+') charIndex = 0;  
        else if (currentChar == '*') charIndex = 1;  
        else if (isdigit(currentChar)) charIndex = 2;  
        else parseError("Invalid character");  
    }  
}
```



```
    action = parseTable[state][charIndex];

    switch (action) {
    case SHIFT:
        printf("Shift: %c\n", currentChar);
        push(&stack, currentChar);
        state = (state + 1) % 5; // Demo transition
        break;

    case REDUCE:
        printf("Reduce by rule\n");
        pop(&stack);
        state = (state - 1 + 5) % 5;
        break;

    case ACCEPT:
        printf("Accepted\n");
        return;

    case ERROR:
    default:
        parseError("Unexpected character or state");
    }
}
```

```
    printf("Parsing complete\n");  
}  
  
int main() {  
    char input[INPUT_SIZE];  
  
    printf("Enter an arithmetic expression: ");  
    fgets(input, INPUT_SIZE, stdin);  
  
    parse(input);  
  
    return 0;  
}
```

Output – 4

```
shiva@Asus MINGW64 /c/Compiiler design Lab
$ gcc Program4.c -o Program4.exe
./Program4.exe
Enter string (use i for id, end with $): i+i+i$
Parsing: i+i+i$
Shift to state 2
Reduce by rule 3
Shift to state 3
Shift to state 4
Reduce by rule 2
Shift to state 3
Shift to state 4
Reduce by rule 2
Accepted
```

Program – 5

Implementation of Operator Precedence Parser

Theory

Aim

To implement an Operator Precedence Parser algorithm for validating arithmetic expressions.

Introduction

An Operator Precedence Parser is a bottom-up parsing technique used in compiler design. It parses expressions based on precedence relations between operators without requiring a full parse tree or grammar derivation.

This parser uses a stack and an input buffer to decide whether to **shift** or **reduce** based on operator precedence.

Role in Compiler Structure

Arithmetic Expression → Operator Precedence Parser → Valid / Invalid Expression

The parser:

- Uses precedence relations to guide parsing
- Performs shift and reduce operations
- Validates syntactic correctness of expressions

Key Concepts

- Uses **stack and input buffer**
- Works on **operator grammar**
- Uses precedence relations:
 - $<\cdot \rightarrow$ Shift

- $= \rightarrow$ Shift
- $\cdot > \rightarrow$ Reduce

ALGORITHM

Step 1: Start

Initialize stack with \$.

Step 2: Input Preparation

Append \$ to the input string.

Step 3: Identify Symbols

Let a be the top terminal of the stack.

Let b be the current input symbol.

Step 4: Apply Rules

- If ($a < \cdot b$ OR $a = b$)
 $\rightarrow$ Shift b to stack
 $\rightarrow$ Move input pointer
- If ($a \cdot > b$)
 $\rightarrow$ Reduce stack symbols
- If ($a = \$$ AND $b = \$$)
 $\rightarrow$ Accept the string
- Otherwise
 $\rightarrow$ Error

Step 5: Repeat

Continue until input is accepted or error occurs.

Step 6: Stop

PSEUDOCODE

1. Push \$ into stack
2. Append \$ to input string
3. Repeat
4. Let a = top terminal of stack

5. Let b = current input symbol
6. if ($a < \cdot b$ or $a = b$)
7. push b into stack
8. move input pointer
9. else if ($a \cdot > b$)
10. pop stack and reduce
11. else if ($a = \$$ and $b = \$$)
12. accept
13. else
14. error

C IMPLEMENTATION

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

#define MAX_INPUT_SIZE 100

const char *input;
char currentChar;

void error(const char *message) {
    printf("Error: %s\n", message);
    exit(1);
}

void nextChar() {
    currentChar = *input++;
}
```

```
void match(char expected) {  
    if (currentChar == expected) {  
        nextChar();  
    } else {  
        error("Unexpected character");  
    }  
}
```

```
void parseExpression();  
void parseTerm();  
void parseFactor();
```

```
void parseExpression() {  
    parseTerm();  
    while (currentChar == '+') {  
        match('+');  
        parseTerm();  
    }  
}
```

```
void parseTerm() {  
    parseFactor();  
    while (currentChar == '*') {  
        match('*');  
        parseFactor();  
    }  
}
```

```
void parseFactor() {  
    if (currentChar == '(') {  
        match('(');  
        parseExpression();  
        match(')');  
    } else if (isdigit(currentChar)) {  
        while (isdigit(currentChar)) {  
            nextChar();  
        }  
    } else {  
        error("Invalid factor");  
    }  
}
```

```
int main() {  
    char inputBuffer[MAX_INPUT_SIZE];  
  
    printf("Enter an arithmetic expression: ");  
    fgets(inputBuffer, sizeof(inputBuffer), stdin);  
  
    input = inputBuffer;  
    currentChar = *input++;  
  
    parseExpression();  
  
    if (currentChar != '\0' && currentChar != '\n') {  
        error("Unexpected characters at the end");  
    }  
  
    printf("Parsing successful\n");  
  
    return 0;  
}  
}
```


OUTPUT – 5

```
PROBLEMS  OUTPUT  DEBUG CONSOLE  TERMINAL  PORTS

shiva@Asus MINGW64 /c/Compiiler design Lab
• $ gcc Program5.c -o compile

shiva@Asus MINGW64 /c/Compiiler design Lab
• $ ./compile
Enter an arithmetic expression: 3+5*2
Parsing successful

shiva@Asus MINGW64 /c/Compiiler design Lab
○ $
```

Program – 6

Implementation of Recursive Descent Parser

THEORY

Aim

To implement a Recursive Descent Parser to validate strings based on a given grammar.

Introduction

Recursive Descent Parsing is a top-down parsing technique in which each non-terminal of a grammar is implemented as a separate procedure. The parser processes the input from left to right and recursively applies grammar rules.

It is one of the simplest parsing techniques and is widely used for small grammars.

Role in Compiler Structure

Token Stream → Recursive Descent Parser → Parse Tree / Validation

The parser:

- Uses recursive procedures
- Matches terminals using functions
- Validates syntax based on grammar

Key Features

- One procedure per non-terminal
- Uses match() for terminals
- Works only for grammars without left recursion
- May require left factoring

- Simple and easy to implement

PREPROCESSING OF GRAMMAR

Before applying recursive descent parsing:

1. Remove Left Recursion

Example:

$E \rightarrow E + T$ (Not allowed)

2. Apply Left Factoring

Used to remove ambiguity and simplify parsing decisions.

ALGORITHM

Step 1: Convert grammar into suitable form
(Remove left recursion and apply left factoring)

Step 2: Take input string w

Step 3: Initialize pointer $pos = 0$

Step 4: Call start symbol procedure $S()$

Step 5: Define procedure for each non-terminal

Step 6: Use $match()$ to verify terminals

Step 7: Use conditional statements for productions

Step 8: Handle ϵ (empty production) if required

Step 9: If mismatch occurs $\rightarrow$ Syntax Error

Step 10: If input fully consumed $\rightarrow$ Accepted, else Rejected

PSEUDOCODE

```
input  $\leftarrow$  string
```

```
pos  $\leftarrow$  0
```

```
procedure match(t):
```

```
    if input[pos] == t then
```

```
        pos  $\leftarrow$  pos + 1
```

```
    else
```

```
        print "Syntax Error"
```

```
        stop
```

```
procedure S():
```

```
    call A()
```

```
procedure A():
```

```
    if input[pos] == 'a' then
```

```
        match('a')
```

```
        call B()
```

```
    else if input[pos] == 'b' then
```

```
        match('b')
```

```
    else
```

```
        print "Syntax Error"
```

```
        stop
```

```
procedure B():  
if input[pos] == 'c' then  
  match('c')  
else  
  print "Syntax Error"  
  stop  
  
procedure parse():  
  call S()  
  if pos == length(input) then  
    print "Accepted"  
  else  
    print "Rejected"
```

PROCEDURE

1. Start the program
2. Input the string
3. Initialize pointer pos = 0
4. Call start symbol function S()
5. Match terminals using match()
6. Follow grammar rules using procedures
7. If mismatch occurs → display Syntax Error
8. If entire string is parsed → Accepted
9. Otherwise → Rejected

C IMPLEMENTATION

```
#include <stdio.h>

#include <string.h>

char input[100];
int pos = 0;

void error() {
    printf("Syntax Error\n");
}

void match(char expected) {
    if (input[pos] == expected) {
        pos++;
    } else {
        error();
    }
}

void B() {
    if (input[pos] == 'c') {
        match('c');
    } else {
        error();
    }
}
```

```
void A() {  
    if (input[pos] == 'a') {  
        match('a');  
        B();  
    } else if (input[pos] == 'b') {  
        match('b');  
    } else {  
        error();  
    }  
}
```

```
void S() {  
    A();  
}
```

```
int main() {  
    printf("Enter string: ");  
    scanf("%s", input);
```

```
    S();
```

```
    if (pos == strlen(input)) {  
        printf("Accepted\n");  
    } else {  
        printf("Rejected\n");  
    }
```

```
    return 0;  
}
```

OUTPUT – 6

```
~/OneDrive/Documents/CD LAB $ gcc program6.c -o program
~/OneDrive/Documents/CD LAB $ ./program
Enter string: ac
Accepted
~/OneDrive/Documents/CD LAB $ |
```


Program – 7

Implementation of Code Optimization Technique

THEORY

Aim

To implement basic code optimization techniques on Three Address Code (TAC).

Introduction

Code Optimization is a phase of a compiler that improves intermediate code to make it more efficient without changing its output. The goal is to reduce execution time and memory usage while preserving correctness.

Optimization works on Three Address Code (TAC) and removes redundancies or simplifies computations.

Role in Compiler Structure

Intermediate Code (TAC) → Code Optimization → Optimized Code → Code Generation

The optimizer:

- Improves performance
- Reduces redundant computations
- Simplifies expressions
- Eliminates unnecessary code

Key Optimization Techniques

- Common Subexpression Elimination
- Constant Folding
- Constant Propagation
- Copy Propagation
- Dead Code Elimination
- Strength Reduction

- Loop Optimization

ALGORITHM

Step 1: Identify Basic Blocks

- Mark first statement as leader
- Mark jump targets as leaders
- Mark statements after jumps
- Form basic blocks

Step 2: Construct Flow Graph

- Represent each block as node
- Add edges based on control flow

Step 3: Optimize Each Basic Block

3.1 Common Subexpression Elimination

Reuse already computed expressions

3.2 Constant Folding

Evaluate constant expressions at compile time

3.3 Constant Propagation

Replace variables with known constant values

3.4 Copy Propagation

Replace variables with assigned values

3.5 Dead Code Elimination

Remove unused computations

3.6 Strength Reduction

Replace costly operations with cheaper ones

3.7 Loop Optimization

Move invariant code outside loops

Step 4: Update Code

Apply all optimizations while maintaining correctness

Step 5: Generate Optimized Code

Output improved TAC

PSEUDOCODE

Input: TAC

Output: Optimized_TAC

1. Identify leaders in TAC

2. Form basic blocks

3. For each block:

 Initialize tables:

 ExprTable, ConstTable, CopyTable

4. For each statement S:

 If $S = x = y \text{ op } z$:

 If $(y \text{ op } z)$ exists:

 replace with previous value

 else store expression

 If constants present:

 evaluate and replace

 If $x = \text{constant}$:

 store in ConstTable

Replace variables using ConstTable

If $x = y$:

store in CopyTable

Replace variables using CopyTable

If result unused:

remove statement

If $x = y * 2$:

replace with $x = y + y$

5. Return Optimized_TAC

PROCEDURE

1. Start the program
2. Input Three Address Code
3. Identify basic blocks
4. Apply optimization techniques
5. Generate optimized code
6. Display result
7. Stop the program

EXAMPLE

Input TAC

$t1 = a + b$

$t2 = a + b$

$t3 = 2 + 3$

$t4 = t3 * c$

$x = t4$

$y = x$

Optimized TAC

$t1 = a + b$

$t2 = t1$

$t3 = 5$

$t4 = 5 * c$

$y = t4$

C IMPLEMENTATION

```
#include <stdio.h>

void constantFolding() {
    int a = 5;
    int b = 10;
    int result = a + b;
    printf("Constant Folding:\n");
    printf("Result of 5 + 10: %d\n", result);
}

void deadCodeElimination() {
    int x = 10;
    int y = 20;

    // Dead code (z is never used)
    int z = x + y;
    int result = x * y;
    printf("Dead Code Elimination:\n");
    printf("Result of 10 * 20: %d\n", result);
}

int main() {
    constantFolding();
    deadCodeElimination();
    return 0;
}
```

OUTPUT -7

```
~/OneDrive/Documents/CD LAB $ gcc program7.c -o program
~/OneDrive/Documents/CD LAB $ ./program
Constant Folding:
Result of 5 + 10: 15
Dead Code Elimination:
Result of 10 * 20: 200
~/OneDrive/Documents/CD LAB $ |
```

Program – 8

Implementation of Code Generator

Theory

Aim

To implement a simple code generator that converts intermediate code (Three Address Code - TAC) into target machine code.

Role in Compiler Structure

Intermediate Code (TAC) → Code Generator → Target Machine Code

The code generator:

- Converts TAC into machine instructions
- Allocates registers
- Emits target code

Key Concepts

- Intermediate Representation (IR / TAC)
- Register Allocation
- Instruction Selection
- Code Emission

ALGORITHM

Step 1: Initialize register set

Step 2: For each instruction in TAC

- Identify operation and operands
- Allocate registers
- Generate instruction
- Store result

- Release registers

Step 3: Output generated code

PSEUDOCODE

```
procedure CODE_GEN(TAC)
begin
  for each instruction instr in TAC do
    begin
      op = instr.operator
      arg1 = instr.arg1
      arg2 = instr.arg2
      result = instr.result

      R1 = getRegister(arg1)
      R2 = getRegister(arg2)

      if op == '+' then emit("ADD R1, R2")
      else if op == '*' then emit("MUL R1, R2")

      assignRegister(result, R1)
      freeRegister(R2)
    end
  end
end
```

PROCEDURE

1. Start the program
2. Input arithmetic expression
3. Parse expression
4. Generate intermediate instructions
5. Display target code
6. Stop

EXAMPLE

Input TAC

t1 = a + b

t2 = t1 * c

Generated Code

MOV R1, a

MOV R2, b

ADD R1, R2

MOV t1, R1

MOV R1, t1

MOV R2, c

MUL R1, R2

MOV t2, R1

C IMPLEMENTATION

```
#include <stdio.h>
#include <stdlib.h>
#include <ctype.h>

void generateCode(const char *instruction) {
    printf("%s\n", instruction);
}

void parseExpression(const char **input);
void parseTerm(const char **input);
void parseFactor(const char **input);

void parseExpression(const char **input) {
    parseTerm(input);
    while (**input == '+') {
        (*input)++;
        parseTerm(input);
        generateCode("ADD");
    }
}

void parseTerm(const char **input) {
    parseFactor(input);
    while (**input == '*') {
        (*input)++;
        parseFactor(input);
        generateCode("MUL");
    }
}

void parseFactor(const char **input) {
    if (isdigit(**input)) {
        printf("LOAD %c\n", **input);
        (*input)++;
    }
}
```

```
else if (**input == '(') {
(*input)++;
parseExpression(input);
if (**input == ')')
(*input)++;
else {
printf("Error\n");
exit(1);
}
}
else {
printf("Error\n");
exit(1);
}
}

int main() {
char input[100];
printf("Enter expression: ");
fgets(input, sizeof(input), stdin);

const char *ptr = input;
parseExpression(&ptr);

return 0;
}
```

OUTPUT – 8

```
~/OneDrive/Documents/CD LAB $ gcc program8.c -o program
~/OneDrive/Documents/CD LAB $ ./program
Enter expression: 7*7+6
LOAD 7
LOAD 7
MUL
LOAD 6
ADD
~/OneDrive/Documents/CD LAB $ |
```

Program – 9

Implementation of Three Address Code with Quadruple Table

Theory

Aim

To implement Three Address Code (TAC) generation and represent it using a Quadruple Table for arithmetic expressions.

Introduction

Three Address Code (TAC) is an intermediate representation used in compilers where each instruction contains at most three addresses. It simplifies complex expressions into a sequence of simple instructions.

Quadruple representation is used to store TAC in a structured form consisting of operator, operands, and result fields.

Role in Compiler Structure

Source Code → TAC → Quadruple Table → Optimization → Code Generation

The TAC:

- Breaks complex expressions
- Uses temporary variables (t1, t2, ...)
- Preserves operator precedence
- Facilitates optimization

Key Points

- Divide expressions into smaller operations
- Use temporary variables
- Follow operator precedence
- Store each instruction in quadruple form

ALGORITHM

1. Start
2. Read input expression exp
3. Initialize:
 - temp = 1
 - index j = 0

Step 1: Handle High Precedence Operators (*, /)

4. Scan expression
5. For each operator:
 - Identify operands
 - Create temp variable (t1, t2, ...)
 - Generate TAC
 - Store in quadruple table
 - Replace expression

Step 2: Handle Low Precedence Operators (+, -)

6. Repeat same process

Step 3: Output

7. Print TAC
8. Print quadruple table
9. Stop

PSEUDOCODE

```
BEGIN
INPUT exp
temp  $\leftarrow$  1
j  $\leftarrow$  0

FOR each operator (*,/)
    generate TAC
    store in table

FOR each operator (+,-)
    generate TAC
    store in table

PRINT quadruple table
END
```

PROCEDURE

1. Start the program
2. Input arithmetic expression
3. Identify operators based on precedence
4. Generate TAC using temporary variables
5. Store instructions in quadruple table
6. Display TAC

7. Display quadruple table
8. Stop the program

EXAMPLE

Input Expression

$a+b*c$

Three Address Code

$t1 = b * c$

$t2 = a + t1$

CIMPLEMENTATION

```
#include <stdio.h>
#include <string.h>

struct quad {
    char op;
    char arg1[10];
    char arg2[10];
    char result[10];
};

int main() {
    char exp[50];
    struct quad q[50];
    int i, j = 0, temp = 1;
    char t[5];

    printf("Enter expression: ");
    scanf("%s", exp);

    int len = strlen(exp);

    // Handle * and /
    for (i = 0; i < len; i++) {
        if (exp[i] == '*' || exp[i] == '/') {
            sprintf(t, "t%d", temp);
            printf("%s = %c %c %c\n", t, exp[i-1], exp[i], exp[i+1]);

            q[j].op = exp[i];
            sprintf(q[j].arg1, "%c", exp[i-1]);
            sprintf(q[j].arg2, "%c", exp[i+1]);
            strcpy(q[j].result, t);
```

```

exp[i+1] = t[1];
temp++;
j++;
}
}

// Handle + and -
for (i = 0; i < len; i++) {
if (exp[i] == '+' || exp[i] == '-') {
sprintf(t, "t%d", temp);
printf("%s = %c %c %c\n", t, exp[i-1], exp[i], exp[i+1]);

q[j].op = exp[i];
sprintf(q[j].arg1, "%c", exp[i-1]);
sprintf(q[j].arg2, "%c", exp[i+1]);
strcpy(q[j].result, t);

exp[i+1] = t[1];
temp++;
j++;
}
}

printf("\nQuadruple Table:\n");
printf("Index\tOp\tArg1\tArg2\tResult\n");

for (i = 0; i < j; i++) {
printf("%d\t%c\t%s\t%s\t%s\n",
i, q[i].op, q[i].arg1, q[i].arg2, q[i].result);
}

return 0;
}

```

OUTPUT – 9

```
~/OneDrive/Documents/CD LAB $ gcc program9.c -o program
~/OneDrive/Documents/CD LAB $ ./program
Enter expression: a*b+c
t1 = a * b
t2 = 1 + c

Quadruple Table:


| Index | Op | Arg1 | Arg2 | Result |
|-------|----|------|------|--------|
| 0     | *  | a    | b    | t1     |
| 1     | +  | 1    | c    | t2     |


~/OneDrive/Documents/CD LAB $ |
```